

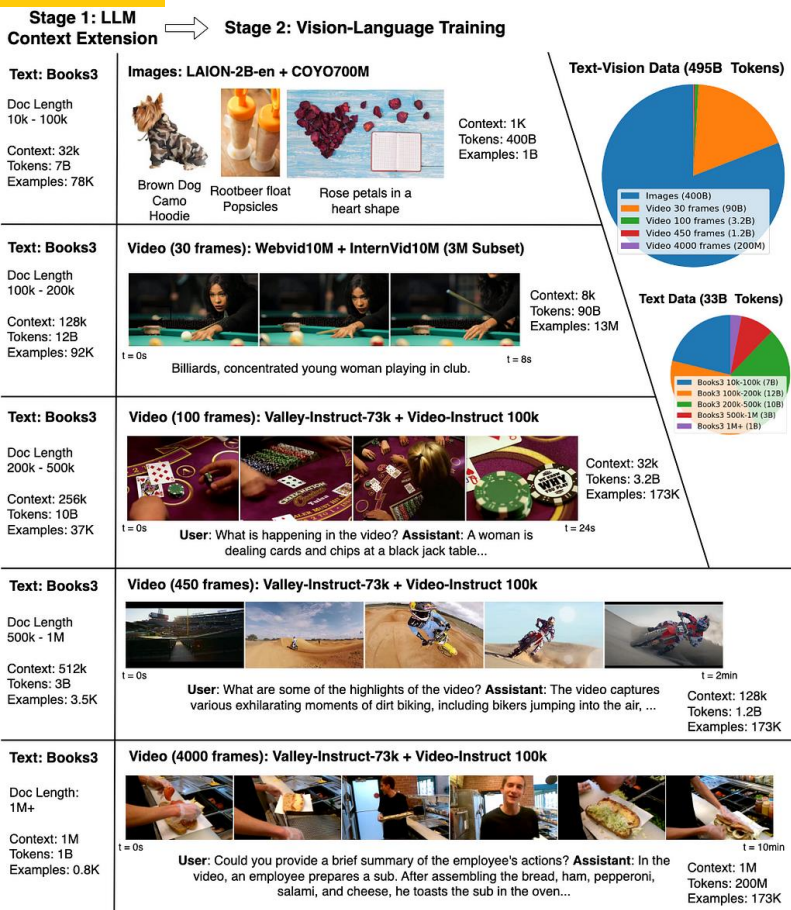
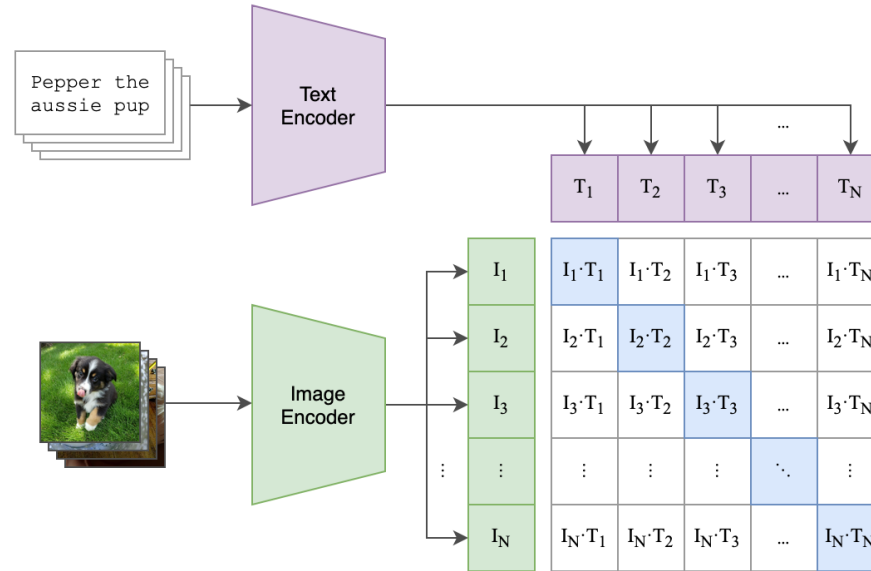


Image Captioning model 동향 조사

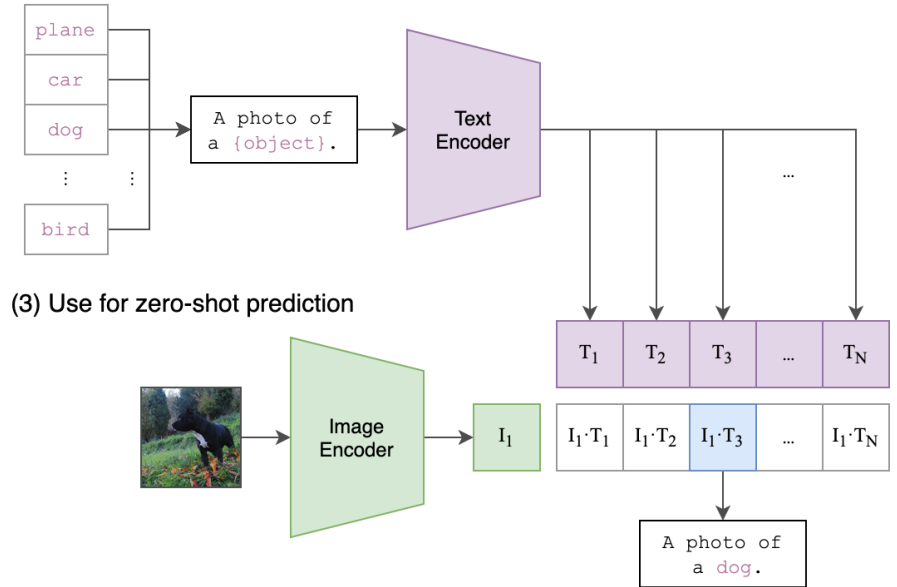
목차

- ▶ 1. CLIP 모델 특징
- ▶ 2. CLIP 모델 활용 image captioning 예시
- ▶ 3. BLIP 모델 특징
- ▶ 4. CLIP/BLIP 모델 활용 image captioning, 성능

(1) Contrastive pre-training



(2) Create dataset classifier from label text



(3) Use for zero-shot prediction

CLIP 모델

ViT와 기본 Transformer 활용 이미지와 텍스트를 동일한 embedding space에 인코딩

→ 이미지와 연관된 텍스트, 그리고 텍스트와 연관된 이미지를 학습

→ 이미지 캡션, classification에 활용 가능

CLIP 활용 image captioning 예시

```
import torch
from PIL import Image
import requests
from transformers import VisionEncoderDecoderModel, ViTImageProcessor, AutoTokenizer
import matplotlib.pyplot as plt

device = "cuda" if torch.cuda.is_available() else "cpu"
print(f"Using device: {device}")

# ViT, GPT2 사용 CLIP 활용
model_id = "ydshieh/vit-gpt2-coco-en"

try:
    # 이미지 로더 및 모델 로드
    model = VisionEncoderDecoderModel.from_pretrained(model_id).to(device)

    # 이미지 처리기, 프로세서
    processor = ViTImageProcessor.from_pretrained(model_id)
    # 텍스트 토크나이저 (토큰, 임베딩, 디코딩)
    tokenizer = AutoTokenizer.from_pretrained(model_id)

    print(f"pre-trained model 로드 성공: {model_id}")

except Exception as e:
    print(f"Error loading model: {e}")
    exit()

def generate_real_caption(image_source):
    print(f"\nGenerating caption for: {image_source}")

    try:
        if image_source.startswith('http'):
            image = Image.open(requests.get(image_source, stream=True).raw).convert("RGB")
        else:
            image = Image.open(image_source).convert("RGB")
    except Exception as e:
        print(f"Error: Could not load image from '{image_source}'. {e}")
        return

    # 이미지 처리기
    pixel_values = processor(images=[image], return_tensors="pt").pixel_values.to(device)

    # 모델 ID 인스턴스 생성
    generated_ids = model.generate(pixel_values, max_length=50)

    # 생성된 ID 인스턴스 -> 토큰으로 변환
    generated_caption = tokenizer.batch_decode(generated_ids, skip_special_tokens=True)[0].strip()

    print(f"\n생성된 image caption: \n")
    print(generated_caption)

    return image

# 이미지 URL
img_url = 'image3.png'
captioned_image = generate_real_caption(img_url)

# 이미지 출력
if captioned_image:
    print(f"\n리미트 이미지 \n")
    plt.imshow(captioned_image)
    plt.title("Input Image")
    plt.axis('off')
    plt.show()
```



생성된 image caption:

a stuffed bear with a cartoon character on it

처리된 이미지



- 캐릭터인 것은 맞으나 생김새가 좀과는 거리가 멀다

배경: 하나의 주제에 특화되지 않고 범용적인(**task-agnostic**) 모델 개발

- ImageNet 과 같은 전통적인 컴퓨터 비전 모델은 사람이 직접 분류한 이미지-텍스트 label 데이터셋으로 모델 학습
- 하지만 이는 많은 양의 학습 데이터 확보에 어려움 존재
- 최근 NLP에서 방대한 주제와 양의 raw text 를 사용하여 모델을 pre-training 한 결과 성능 대폭 향상
- CLIP의 핵심 아이디어는 이러한 NLP 접근법을 CV에 적용. 이미지-텍스트 label 데이터셋 대신, 이미지-raw text 데이터로 학습

CLIP 모델 특징



높은 ¹⁾zero-shot classification 성능

범용성 높은 Pre-training 기법 적용
→ 하나의 이미지 종류만 학습한 모델보다 거의 모든 카테고리에서 우월한 성능



학습 데이터 확보 용이

사람이 수동으로 이미지에 대한 정답 label을 제공할 필요 없이 raw text로 학습 가능



“multimodal” embedding: 이미지와 텍스트의 관계 학습

동일한 embedding space 에서 이미지: ViT와 텍스트: Transformer의 attention 메커니즘으로 이미지-텍스트 간 복잡한 관계 학습 가능



높은 학습 효율 및 범용성

적은 양의 학습 데이터로 타 모델 대비 높은 성능 보임

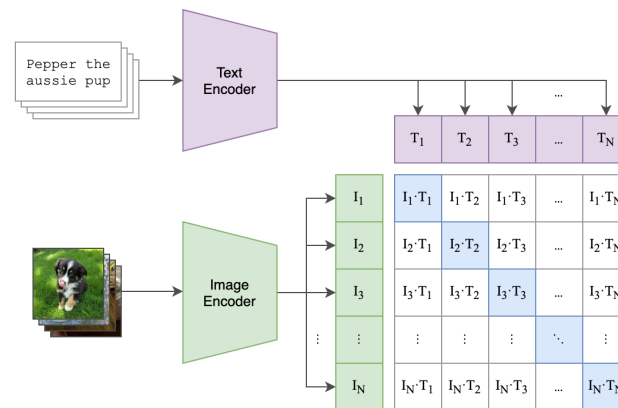
¹⁾ 학습하지 않은 이미지에 대한 분류 정확도

CLIP 모델 예시 코드

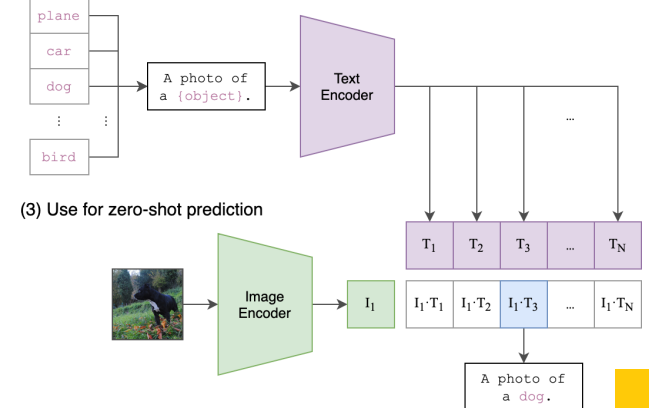
```
# image_encoder - ViT
# text_encoder - Text Transformer
# I[n, h, w, c] - minibatch of aligned images
# T[n, l] - minibatch of aligned texts
# W_i[d_i, d_e] - learned proj of image to embed
# W_t[d_t, d_e] - learned proj of text to embed
# t - learned temperature parameter
# extract feature representations of each modality
I_f = image_encoder(I) # [n, d_i]
T_f = text_encoder(T) # [n, d_t]
# 인코딩 된 데이터를 embedding space로 projection,
# [n, d_e]
I_e = l2_normalize(np.dot(I_f, W_i), axis=1)
T_e = l2_normalize(np.dot(T_f, W_t), axis=1)
# scaled (image, text) cosine similarities
# in the embedding space
# [n, n]
logits = np.dot(I_e, T_e.T) * np.exp(t)
# contrastive loss
labels = np.arange(n)
loss_i = cross_entropy_loss(logits, labels, axis=0)
loss_t = cross_entropy_loss(logits, labels, axis=1)
loss = (loss_i + loss_t)/2
```

1. 미리 학습된 이미지, 텍스트 인코더 사용
2. 길이 n의 미니배치로 이미지와 텍스트를 나눔
3. 인코더의 아웃풋을 1) embedding space에 2) projection
4. Normalize 후 cosine similarity 계산 (각 벡터 dot product)
5. Contrastive loss 계산

(1) Contrastive pre-training



(2) Create dataset classifier from label text



(3) Use for zero-shot prediction

- 1) 각 이미지-텍스트 쌍의 유사도가 계산되는 벡터 공간
- 2) embedding space 차원에 맞도록, 학습된 행렬을 곱셈

Contrastive Loss

Contrastive Supervised Approach



Image Encoder

Update to make matching pairs 100% and negative pairs 0%

90% match	0% match	0% match	20% match	1% match
0% match	98% match	0% match	3% match	7% match
10% match	2% match	99% match	0% match	2% match
5% match	0% match	0% match	97% match	1% match
0% match	1% match	0% match	1% match	93% match

Text Encoder

A picture of a dog wearing a fabulous jacket
A cat lying down
A Cool Photo of a Lizard
Skate Boarding With Dog
Skate Boarder

Update to make matching pairs 100% and negative pairs 0%

Cosine similarity matrix 에서

- 대각선 $\langle x_i, y_i \rangle$: 정답 (이미지, 텍스트) 쌍
- 나머지 $\langle x_i, y_j \rangle, i \neq j$: 오답 (이미지, 텍스트) 쌍

$$\mathcal{L} = -\frac{1}{2N} \sum_{i=1}^N \left(\overbrace{\log \frac{e^{t\mathbf{x}_i \cdot \mathbf{y}_i}}{\sum_{j=1}^N e^{t\mathbf{x}_i \cdot \mathbf{y}_j}}}^{\text{image} \rightarrow \text{text softmax}} + \overbrace{\log \frac{e^{t\mathbf{x}_i \cdot \mathbf{y}_i}}{\sum_{j=1}^N e^{t\mathbf{x}_j \cdot \mathbf{y}_i}}}^{\text{text} \rightarrow \text{image softmax}} \right)$$

Every positive pair is normalized by all negative pairs

- 대각선 $\langle x_i, y_i \rangle$ logit값에 대한 softmax 적용
- Loss 최소화 $\leftrightarrow$ 대각선 $\langle x_i, y_i \rangle$ logit 값 최대화
= image ¹⁾ cross entropy loss + text cross entropy loss
→ 정답 쌍은 가까워지고 오답 쌍은 멀어지도록 유도

¹⁾ 정답값과 예측값을 입력으로 받으며 정답값 = 예측값 일때 값이 가장 작은 함수.
분류 모델 학습에 쓰임

ClipClap: CLIP + GPT 활용 캡션 생성

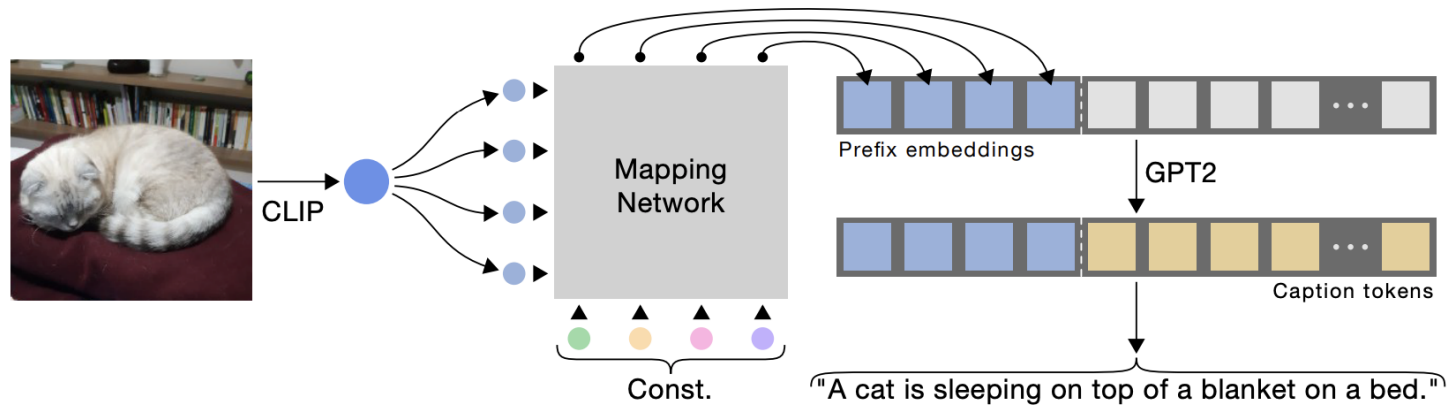
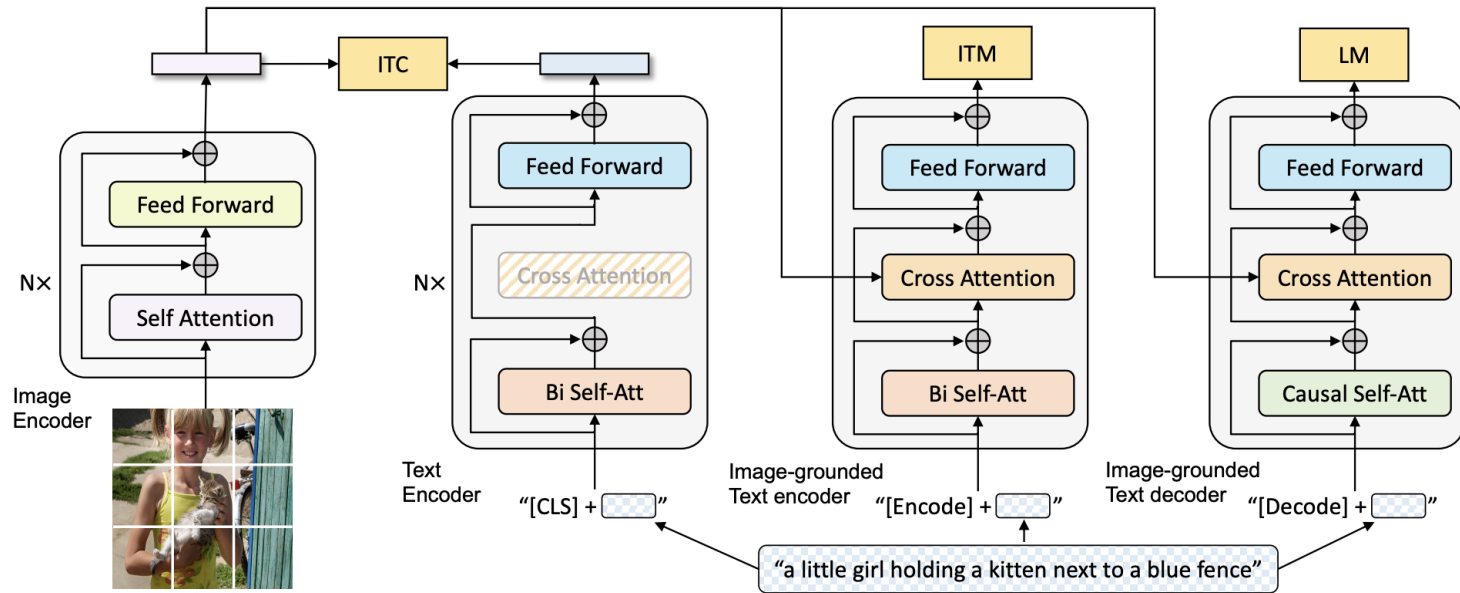


Figure 2. Overview of our transformer-based architecture, enabling the generation of meaningful captions while both CLIP and the language model, GPT-2, are frozen. To extract a fixed length prefix, we train a lightweight transformer-based mapping network from the CLIP embedding space and a learned constant to GPT-2. At inference, we employ GPT-2 to generate the caption given the prefix embeddings. We also suggest a MLP-based architecture, refer to Sec. 3 for more details.

- CLIP image encoder의 아웃풋을 ¹⁾ Mapping network(MLP) 를 통해 GPT2로 전달
 - 이미지-텍스트를 통해 인코더가 학습되었으므로 image captioning에 매우 적합
- 캡션 텍스트 생성은 GPT2가 담당

¹⁾ 범용성이 뛰어나며 기본적인 인공지능망



BLIP: 더 강력한 VLP(vision-language pretraining) 모델

기존 CLIP모델: 인터넷 상의 image-text 데이터를 크롤링 하는 과정에서 퀄리티가 떨어지는 noise 데이터 존재

BLIP: ClapFilt 적용으로 데이터셋 퀄리티 향상, 새로운 모델 아키텍처 사용

BLIP 활용 image captioning 예시

```
import torch
from PIL import Image
import requests
from transformers import AutoProcessor, BlipForConditionalGeneration
import matplotlib.pyplot as plt

device = "cuda" if torch.cuda.is_available() else "cpu"
torch_dtype = torch.float16 if torch.cuda.is_available() else torch.float32
print(f"Using device: {device}")

# BLIP
model_id = "Salesforce/blip-image-captioning-large"

try:
    # VLT 모델 불러오기 (모델 ID 지정)
    processor = AutoProcessor.from_pretrained(model_id)

    # BLIP 모델
    model = BlipForConditionalGeneration.from_pretrained(
        model_id,
        torch_dtype=torch_dtype
    ).to(device)

    print("BLIP model loaded successfully!")
except Exception as e:
    print(f"Error loading model: {e}")
    exit()

def generate_caption(image_source):
    """ 이미지 캡션 """
    print(f"Generating caption for: {image_source}")
    try:
        if image_source.startswith('http'):
            raw_image = Image.open(requests.get(image_source, stream=True).raw).convert('RGB')
        else:
            raw_image = Image.open(image_source).convert('RGB')
    except Exception as e:
        print(f"Error: Could not load image from '{image_source}'. {e}")
        return None

    # 이미지 처리
    inputs = processor(images=raw_image, return_tensors="pt").to(device, torch_dtype)

    # BLIP 모델에 입력된 데이터를 처리
    generated_ids = model.generate(**inputs, max_length=50)

    # 생성된 ID를 텍스트로 변환
    generated_caption = processor.batch_decode(generated_ids, skip_special_tokens=True)[0].strip()

    print(f"\n생성된 image caption: ")
    print(generated_caption)

    return raw_image

# 이미지 경로
img_path = 'image3.png'
captioned_image = generate_caption(img_path)

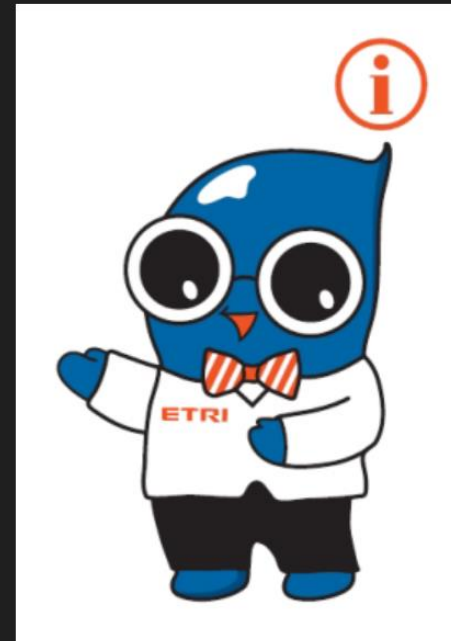
# 이미지 출력
if captioned_image:
    print(f"\n처리가된 이미지\n")
    plt.imshow(captioned_image)
    plt.axis('off') # Hide the axes for a cleaner look
    plt.show()
```



생성된 image caption:

cartoon character of a blue owl wearing a tie and a white shirt

처리된 이미지



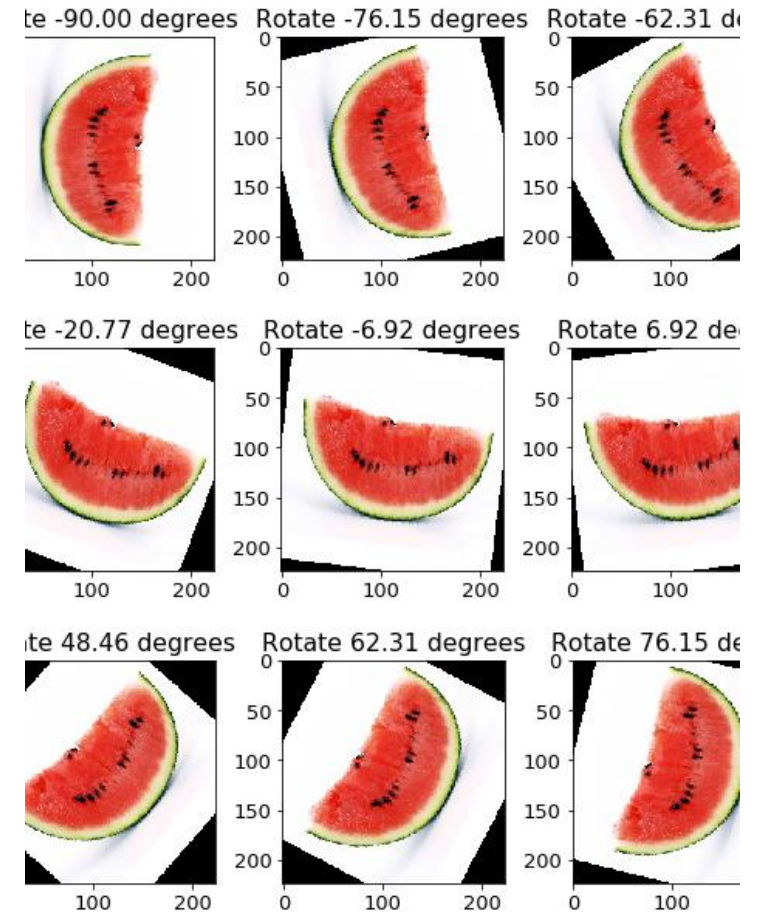
- ✓ CLIP기반 모델보다 성능이 향상됨
- ✓ 구체성: “넥타이”, “흰 셔츠”
- ✓ 곰 보다는 푸른 부엉이에 더 근접함

Knowledge Distillation (KD)

- KD: 선생님 모델(크고 강력) 이 학생 모델(작고 효율적) 로 아웃풋을 전달함
- 기존 KD 기법은 단순히 학생 모델이 선생님 모델과 같은 아웃풋을 내도록 훈련시킴
- BLIP에서는 CapFilt(캡션 필터) 기법 사용
 - 선생님 모델: captioner는 고퀄리티의 캡션을 생성
 - 학생 모델: filter는 선생님 모델로부터 받은 캡션을 필터링함

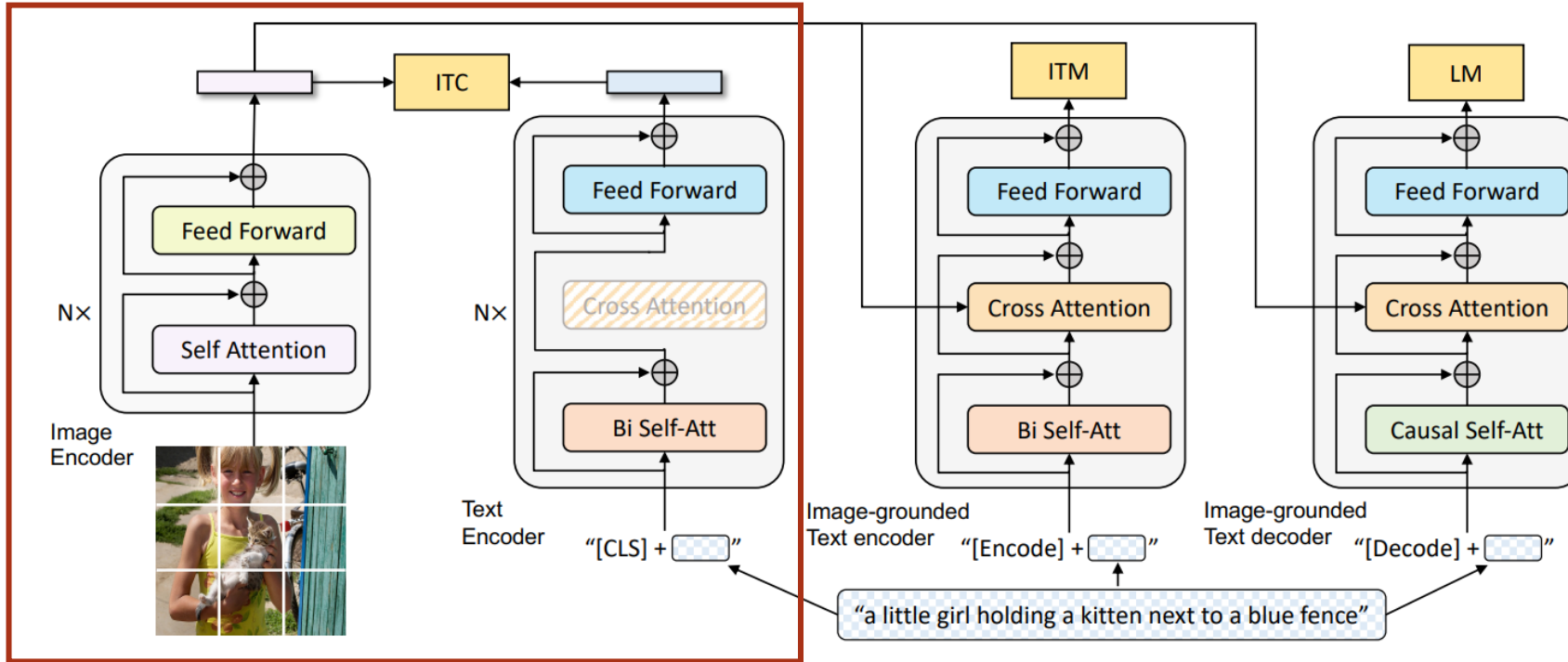
1) Data Augmentation

- NLP에 적용되는 Data Augmentation은 이미지에 적용되는 증강 (회전, 좌우반전, 사이즈 조정,...) 보다 직관적이지 못함
- 하지만 최근 NLP에도 생성형 모델로 인공적인 텍스트를 만들어 데이터 증강을 적용하고 있음
- BLIP모델은 단순한 language-only task를 넘어 Vision-Language task에도 인공적인 텍스트 데이터셋을 통한 데이터 증강이 효과적임을 보여줌



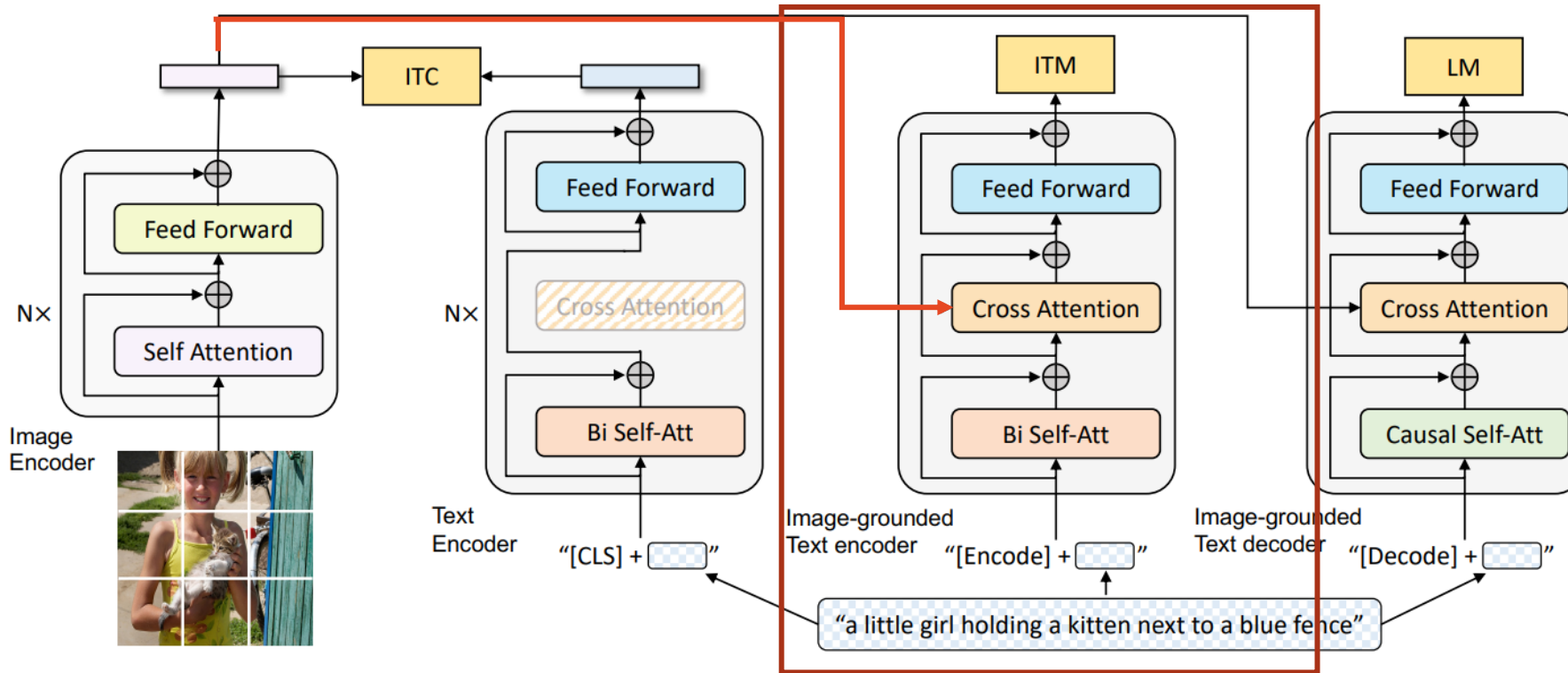
1) 학습 과정에서 모델이 단순히 훈련 데이터를 외우는 것을 방지하기 위해 일부러 훈련 데이터에 변화를 주는 기법. 모델의 일반화 성능 향상: 훈련 데이터와 비슷하지만 조금씩 다른 입력에 대해서도 올바른 출력을 내도록 함

모델 아키텍처 (1) unimodal encoder



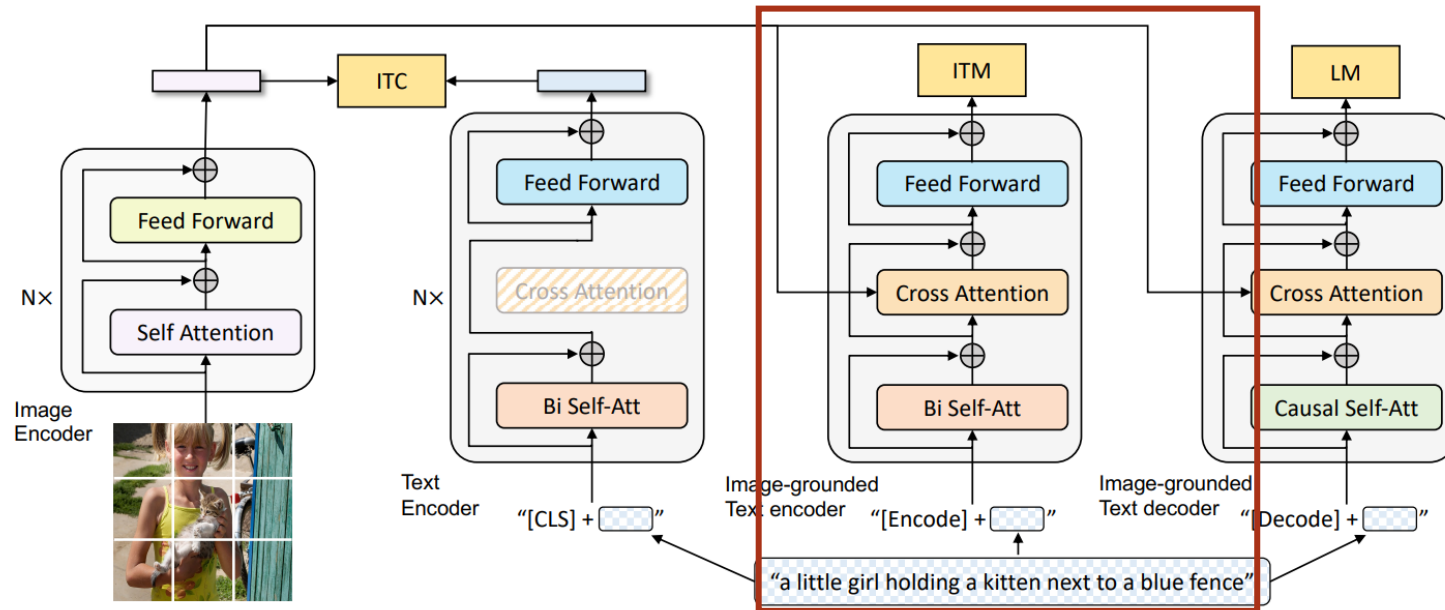
- 1) ViT, 2) BERT 사용 이미지와 텍스트를 따로 인코딩
 - 전체 문장의 정보가 요약되도록 문장 앞에 '[CLS]' 토큰 삽입
- 목표: ViT와 BERT의 아웃풋 (feature 벡터)이 정답 이미지-텍스트 쌍에 대하여 비슷하고, 오답 이미지-텍스트 쌍에 대하여 멀어지도록 함
- CLIP과 마찬가지로 Contrastive Loss 사용

모델 아키텍처 (2) Image-grounded text encoder



- 정답 이미지-텍스트 쌍 학습
- 마찬가지로, 문장 앞에 '[Encode]' 토큰을 추가하여 문장을 요약하는 첫 번째 벡터 생성
- BERT 인코더, cross attention 사용으로 이미지 기반 이미지-텍스트 쌍의 representation 학습
- Image-Text Matching Loss(ITM)

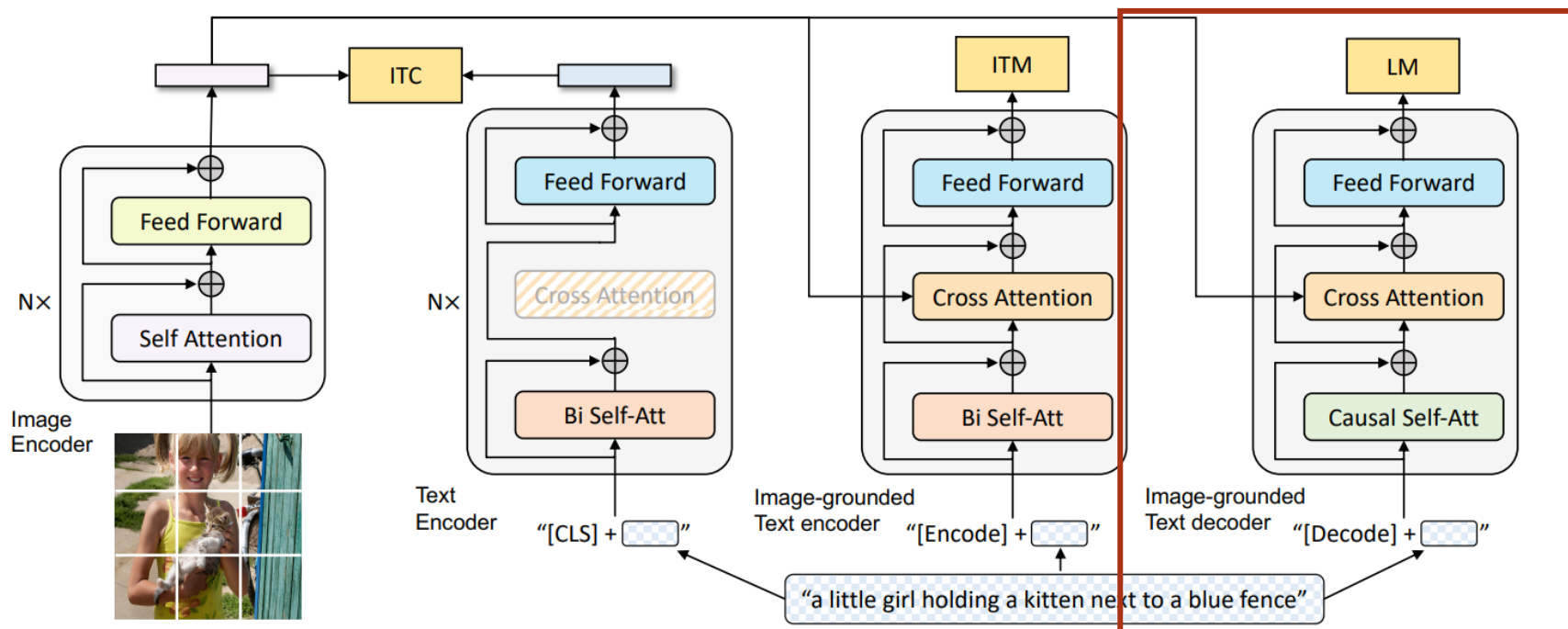
ITM Loss for (2) Image-grounded text encoder



$$\mathcal{L}_{itm} = \mathbb{E}_{(I,T) \sim D} H(y^{itm}, p^{itm}(I, T))$$

- 이진 분류 수행. $p(I,T)$: 이미지, 텍스트 쌍이 주어졌을 때 이것이 positive(정답) 쌍 인지 또는 negative(오답) 쌍 인지 예측, softmax 적용
- $y(I, T)$: 정답 레이블 [0, 1] or [1,0], Cross Entropy Loss 사용
- “**Hard negative mining**”: 오답이면서 유사도가 높은 쌍들을 어려운 오답(hard negative)으로 정의. 각 이미지마다 모든 이미지-텍스트 쌍에 대한 Loss를 계산하지 않고 하나의 hard negative 쌍에 대하여 ITM Loss 계산

모델 아키텍처 (3) Image-grounded Text Decoder



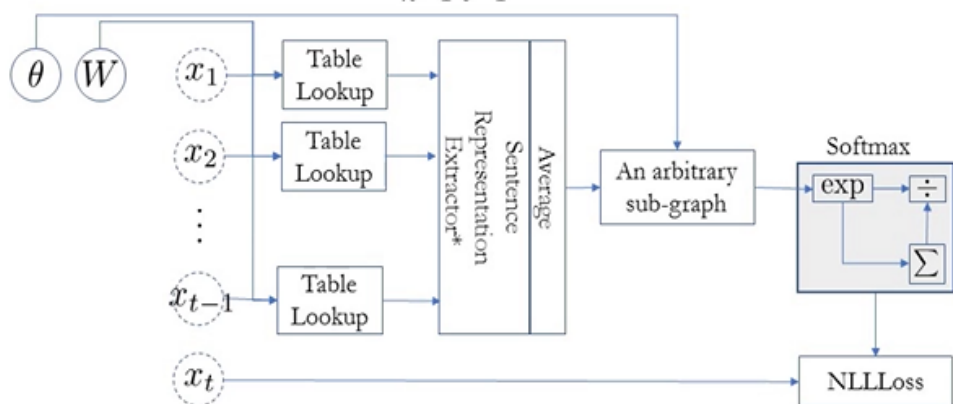
- 문장 앞에 '[Decode]' 토큰을 추가하여 문장 전체를 요약하는 첫번째 아웃풋 벡터 생성
- BERT가 아닌 causal self attention을 사용 (e.g. GPT)
- 1) cross attention을 통해 이미지에 기반한 텍스트 생성
- Language Modeling Loss (LM)

LM Loss for (3) Image-grounded text decoder

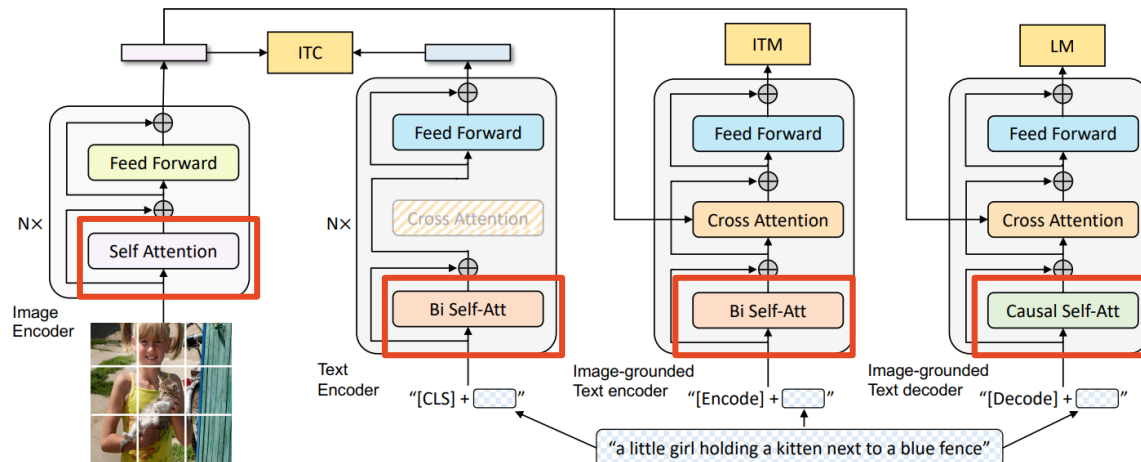
Autoregressive language modelling

- Autoregressive sequence modelling $p(X) = \prod_{t=1}^T p(x_t|x_{<t})$
- Loss function: the sum of negative log-probabilities

$$\log p_{\theta}(X) = \sum_{n=1}^N \sum_{t=1}^T \log p_{\theta}(x_t|x_{<t})$$



- 일반적인 언어 모델의 autoregressive loss 사용
 - 생성된 token t 마다 모델 아웃풋(softmax)과 정답 분포에 대하여 Cross Entropy Loss 계산
- 1) Self-Attention이 encoding과 decoding의 차이를 가장 많이 담고 있으므로,
- Parameter sharing: Encoder, decoder Self-attention을 제외한 cross attention, FFN 끼리는 weights 공유 효율 향상



1) 하나의 텍스트나 이미지 토큰들 사이의 관계와 의미 학습

CapFilt: 더 정확한 훈련 데이터 캡션 생성

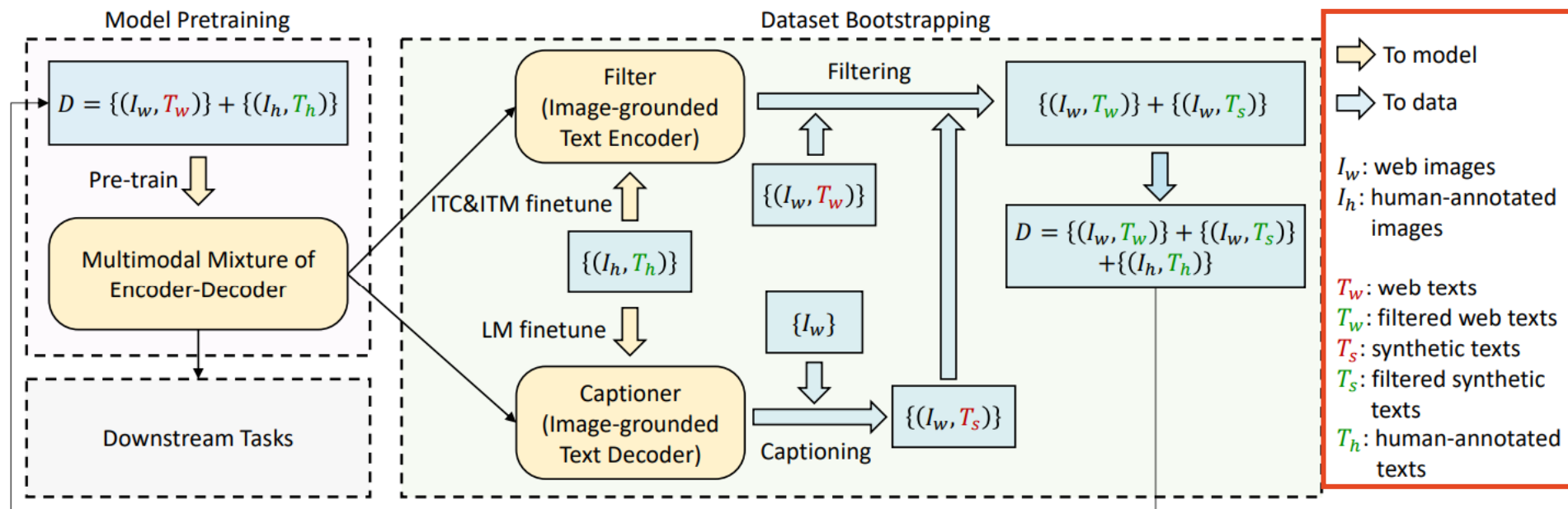


Figure 3. Learning framework of BLIP. We introduce a captioner to produce synthetic captions for web images, and a filter to remove noisy image-text pairs. The captioner and filter are initialized from the same pre-trained model and finetuned individually on a small-scale human-annotated dataset. The bootstrapped dataset is used to pre-train a new model.

- Bootstrapping을 통해 **인공적** 캡션 T_s , 사람이 생성한 캡션 T_h , 웹 크롤링 캡션 T_w 필터링 → 데이터셋의 양과 질 향상
- (2) Image-grounded Text Encoder과 (3) Image-grounded Text Decoder를 사람이 작성한 캡션 데이터로 파인튜닝 후 bootstrapping

CapFilt 적용 결과



T_w : "from bridge near my house"

T_s : "a flock of birds flying over a lake at sunset"



T_w : "in front of a house door in Reichenfels, Austria"

T_s : "a potted plant sitting on top of a pile of rocks"



T_w : "the current castle was built in 1180, replacing a 9th century wooden castle"

T_s : "a large building with a lot of windows on it"

- 이미지 캡션 정확도 향상

Pre-train dataset	Bootstrap		Vision backbone	Retrieval-FT (COCO)		Retrieval-ZS (Flickr)		Caption-FT (COCO)		Caption-ZS (NoCaps)	
	C	F		TR@1	IR@1	TR@1	IR@1	B@4	CIDEr	CIDEr	SPICE
COCO+VG+CC+SBU (14M imgs)	✗	✗	ViT-B/16	78.4	60.7	93.9	82.1	38.0	127.8	102.2	13.9
	✗	✓ _B		79.1	61.5	94.1	82.8	38.1	128.2	102.7	14.0
	✓ _B	✗		79.7	62.0	94.4	83.6	38.4	128.9	103.4	14.2
	✓ _B	✓ _B		80.6	63.1	94.8	84.9	38.6	129.7	105.1	14.4
COCO+VG+CC+SBU+LAION (129M imgs)	✗	✗	ViT-B/16	79.6	62.0	94.3	83.6	38.8	130.1	105.4	14.2
	✓ _B	✓ _B		81.9	64.3	96.0	85.0	39.4	131.4	106.3	14.3
	✓ _L	✓ _L		81.2	64.1	96.0	85.5	39.7	133.3	109.6	14.7
	✗	✗	ViT-L/16	80.6	64.1	95.1	85.5	40.3	135.5	112.5	14.7
	✓ _L	✓ _L		82.4	65.1	96.7	86.7	40.4	136.7	113.2	14.8

Table 1. Evaluation of the effect of the captioner (C) and filter (F) for dataset bootstrapping. Downstream tasks include image-text retrieval and image captioning with finetuning (FT) and zero-shot (ZS) settings. TR / IR@1: recall@1 for text retrieval / image retrieval. ✓_{B/L}: captioner or filter uses ViT-B / ViT-L as vision backbone.

CLIP/BLIP 기반 image captioning 성능 향상 방법

1. 고품질, 대량의 데이터셋 사용
2. 데이터 증강 (image data augmentation)
3. 고성능의 pre-trained model 사용
4. ¹⁾ LoRA 활용 Finetuning
5. 하이퍼파라미터 튜닝
6. Learning rate scheduling

¹⁾ 모델 각 레이어의 일부 가중치만 학습하여 효율적으로 파인튜닝 시키는 기법

BLIP 기반 image captioning 성능 향상 예시: LoRA



what is this dont us einternet



Show thinking ▼

Based on the image, this is a close-up of a **traditional Korean wooden veranda or porch**, known as a **maru** (마루).

Finetuning 주제: 한옥 (“hanok”)

- 최신 유료 모델인 Google의 Gemini2.5 Pro조차 인터넷 검색 없이 **정답: 계자난간** 을 맞히지 못함
- 한옥 계자난간에 대한 1300 쌍 의 training image 와 JSON 메타데이터, 그리고 170쌍의 validation image 와 JSON 메타데이터를 가지고 모델 finetuning 실행

Finetuning with LoRA

```
"metadata": {  
  "metadata.area": "Seongju",  
  "metadata.location": "hangae2gil",  
  "metadata.cultural_heritage": "Nal",  
  "metadata.hanok_type": "diguedja",  
  "metadata.hanok_house": "anchae",  
  "metadata.weather": "clear",  
  "metadata.category": "railing",  
  "metadata.motive_pattern": "gyejanangan",  
  "metadata.shooting_direction": "left",  
  "metadata.shooting_height": "low"  
}
```

전통가옥 학습 데이터에서 JSON 메타 데이터를
오픈소스 LLM으로 추출하여 각 이미지 별 정답
캡션을 만듦

이미지-캡션 데이터를 가지고 LoRA를 활용하여
BLIP모델의 q, k projection layer를 파인튜닝

```
prompt_template = f"""  
You are an expert in describing Korean architecture.  
Based on the following metadata, create a single, natural, and descriptive English sentence for an image caption.  
  
Metadata:  
- Area: {metadata['area']}  
- Location: {metadata['location']}  
- Hanok Type: {metadata['hanok_type']} shape  
- Hanok Part: {metadata['hanok_house']}  
- Weather: {metadata['weather']}  
- Category: {metadata['category']} view  
- Shooting Direction: from the {metadata['shooting_direction']}  
- Shooting Height: from a {metadata['shooting_height']} angle  
  
Generated Caption:  
"""
```

```
config = LoraConfig(  
    r=16,  
    lora_alpha=32,  
    lora_dropout=0.05,  
    target_modules=["q_proj", "k_proj"],  
    bias="none",  
)
```

Finetuning with LoRA 결과: 학습하지 않은 계자난간 이미지 캡션 생성

Image: img1.jpg

Generated Caption: this is a gyejanangan, a traditional korean railing, featured on a ilja - shaped ancha person and a clear

Caption: this is a gyejanangan, a traditional korean railing, featured on a ilja - shaped ancha person and a clear



- 오픈소스 가벼운 LM의 한계로 문장이 매끄럽지는 못하나, 계자난간(“gyejanangan”)이라는 것을 학습해내어 캡션 생성

Image: sou_H003_sj_anchae_clear_nal_railing_gyejanangan_nanganfr1_rhig_P000493.jpg

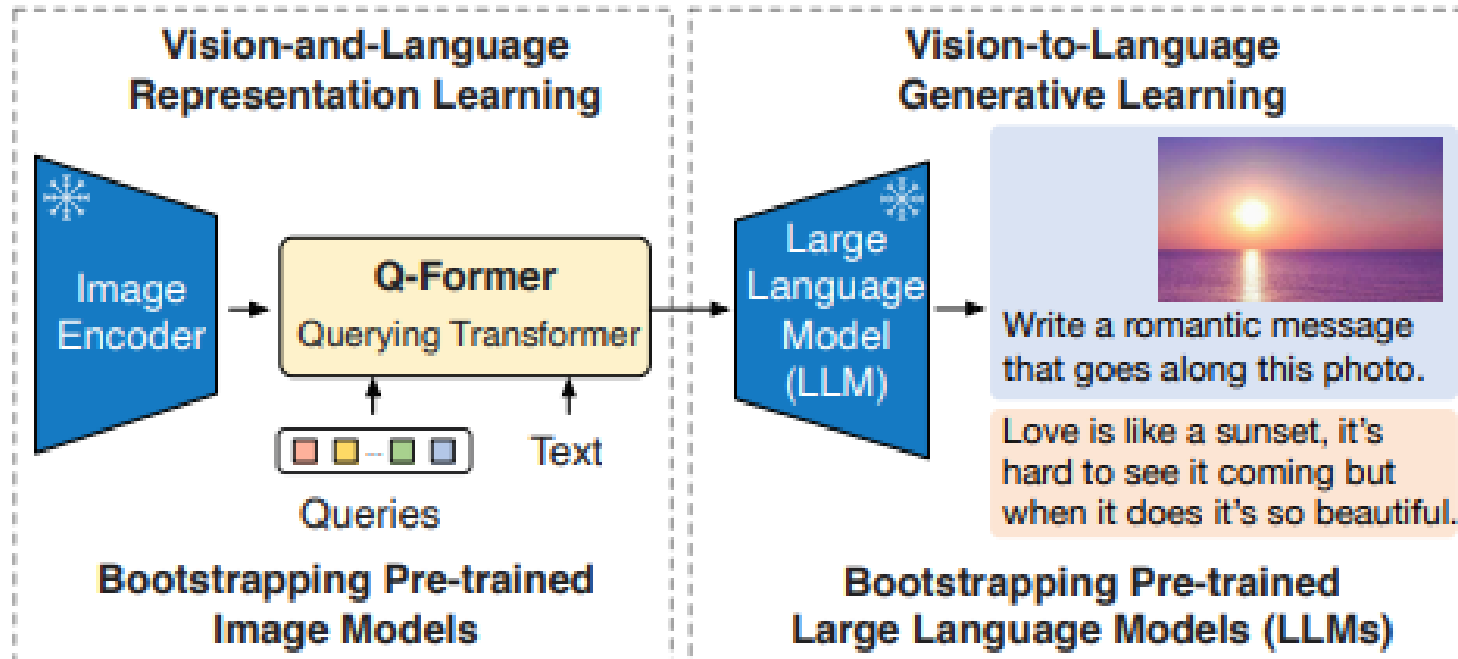
Generated Caption: this is a gyejanangan, a traditional korean railing, featured on

Caption: this is a gyejanangan, a traditional korean railing, featured on a diguedja - shaped ancha - shaped the photo was taken on a clear day from a high angle, looking from the right the right the right



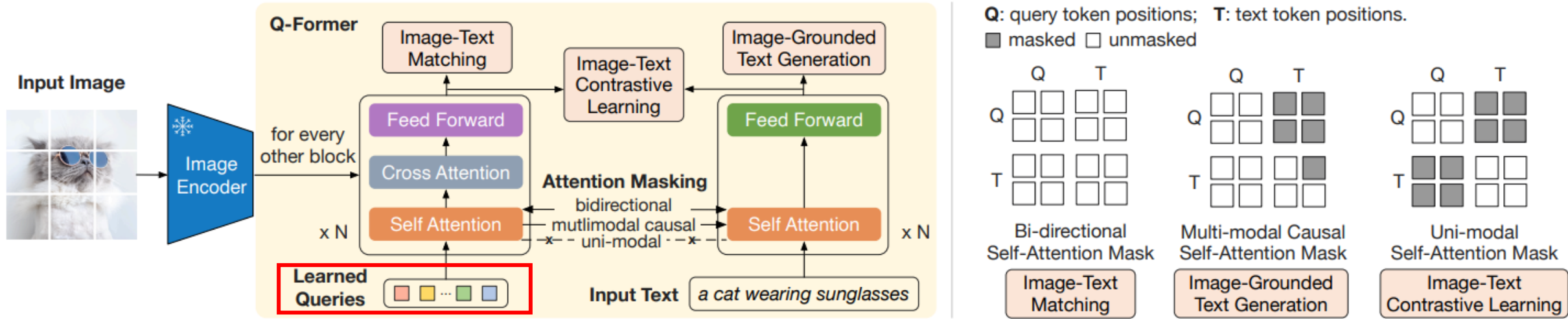
- 디귤자 모양 한옥 안채, 맑은 날씨, 높은 각도, 오른쪽에서 찍은 사진이라는 캡션도 생성해냄
- 학습 데이터가 많아진다면 정확도가 더 높아질 것임

최신 **BLIP** 모델: **BLIP - 2**



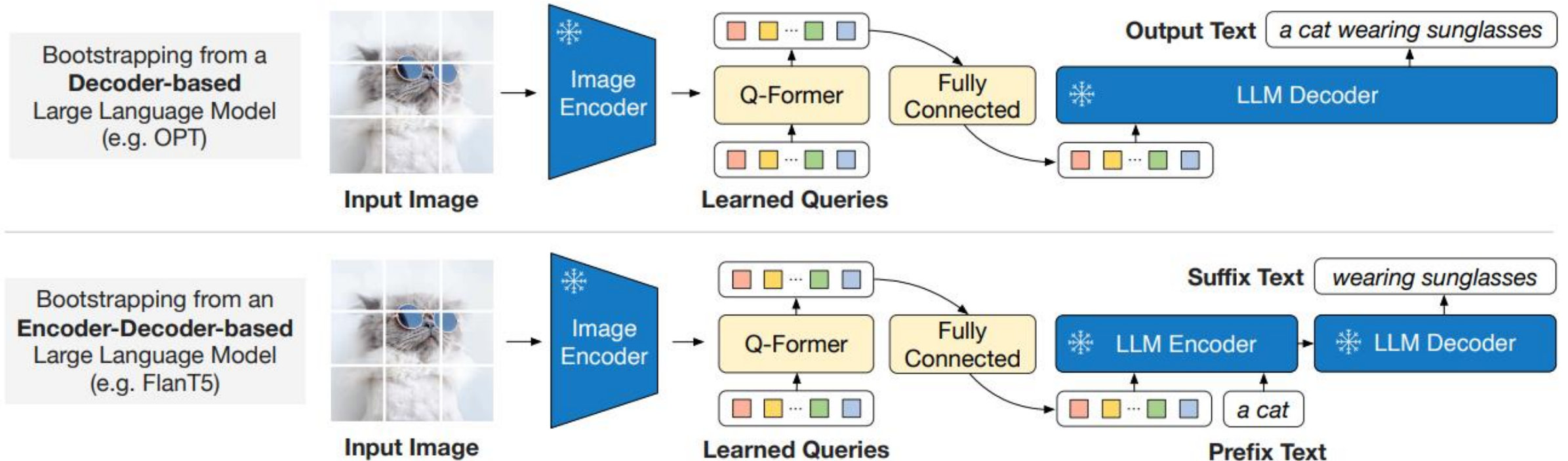
- **핵심 아이디어:** 기존 **unimodal** 이미지 인코더와 LLM은 그대로 사용하면서 이미지 캡셔닝 성능 향상
- 1단계: 이미지의 표현 학습 (**이미지 인코더**) → 2단계: 이미지 표현에 기반한 텍스트 생성 학습 (**LLM**)
- 학습 효율을 위해 이미지 인코더와 LLM의 가중치는 고정시킨 상태에서, **Q-former**만 학습시킴

Q – Former: 모달리티 간극 해소

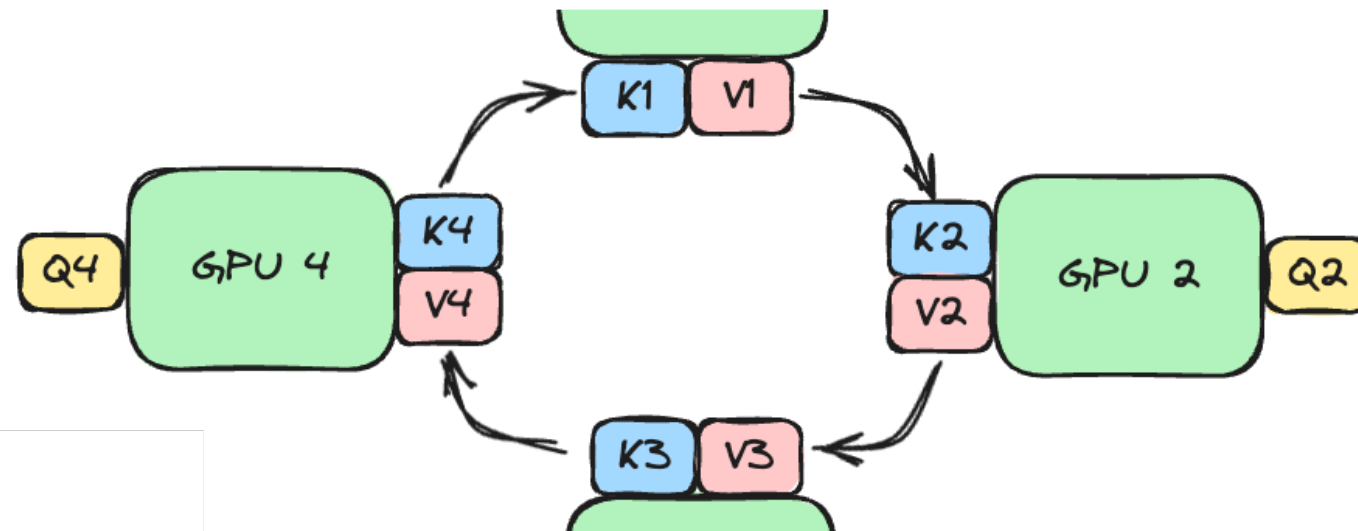


- 학습 stage 1: 기존 BLIP 모델과 동일한 3가지 목적 함수가 사용됨. 목적에 맞는 각기 다른 mask 사용
- Q – former는 **Query** - Transformer의 약자임
- Q – Former의 역할: 가중치가 고정된 unimodal **이미지** 인코더와 **LLM** 사이의 **modality** 간극 해소
- “**Queries**”란 어텐션 메커니즘의 Q가 아닌, 학습 가능한 가중치로써 self-attention과 cross-attention을 통해 텍스트-이미지 관계와 표현 학습
 - 처음에는 무작위로 시작되지만 학습을 통해 **Queries**의 값들이 점차 텍스트-이미지 관계, 표현 학습

Q – Former: Unimodal encoders 연결



- 학습 stage 2: stage 1에서 학습한 Q-former의 아웃풋을 가지고 텍스트 생성 학습
- Fully connected layer을 가지고 Q-former 아웃풋과 LLM 연결



Blockwise Ring Attention

기존 어텐션 메커니즘의 메모리 복잡도 문제 해결



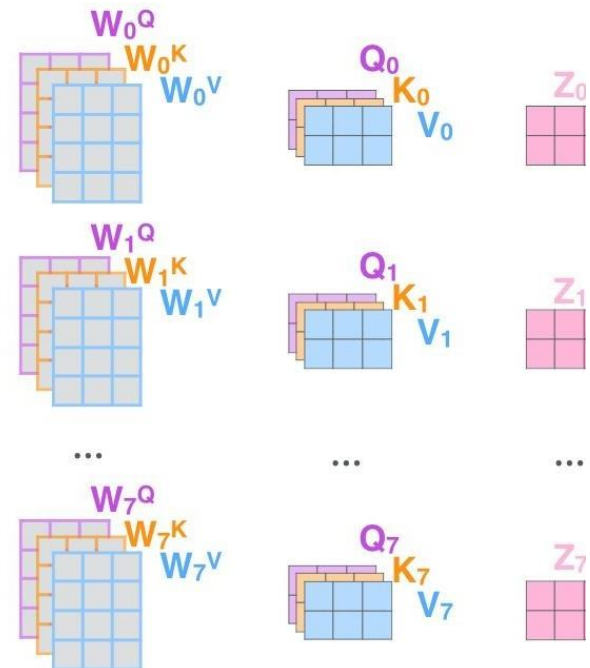
기존 Transformer의 문제점: 컴퓨팅 효율

- 기존 Transformer의 self-attention 메커니즘은 행렬 곱셈 과정에서 인풋의 길이가 n 만큼 증가시 연산시간과 메모리 사용량은 n^2 만큼 증가
- 메모리 용량의 한계 때문에 넓은 context window를 가진 모델 학습에 어려움 존재
- 그래서 긴 동영상이나 글과 사진을 이해하는 데 한계가 있었음

3) Split into 8 heads.
We multiply X or R with weight matrices

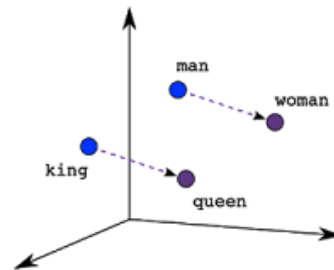
4) Calculate attention
using the resulting $Q/K/V$ matrices

5) Co
then
prod

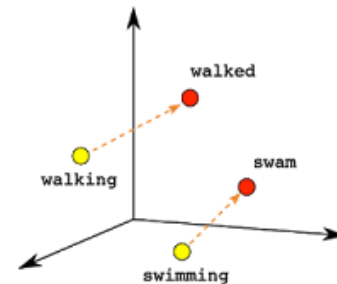


Self-Attention 메커니즘

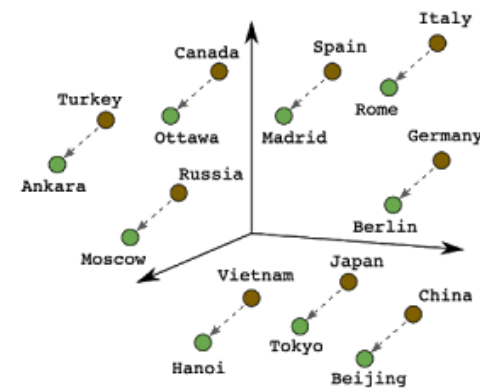
$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$$
$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$$



Male-Female



Verb Tense



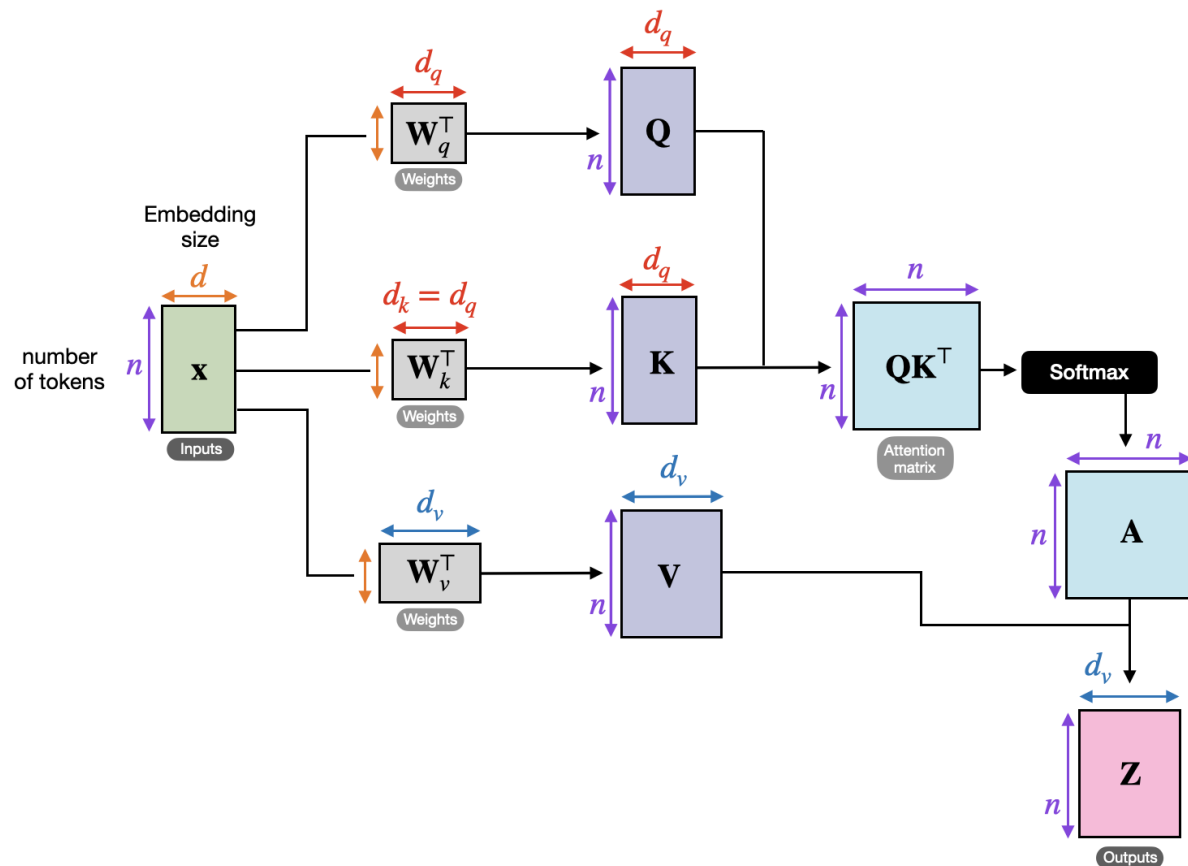
Country-Capital

- 먼저 인풋 텍스트를 tokenizer를 통해 토큰화 e.g. ["[문장 시작]", "I", "am", "happy", "[문장 끝]"]
- 그 다음 각 token을 $1 \times d$ 의 벡터로 embedding한다. 임베딩 공간에서 비슷한 의미를 가진 토큰은 거리가 가까움
- 토큰의 문장 내 위치 정보를 담은 $1 \times d$ 의 positional encoding 벡터를 임베딩 벡터에 더해 positional encoding을 수행
 - PE(pos: 문장 내 토큰의 위치, i: embedding dimension d 안에서의 위치)
- 이 positional encoding까지 완료된 인풋 토큰 행렬 ($n \times d$) 를 **X** 라고 한다

Self-Attention 메커니즘

$$Attention(Q, K, V) = softmax\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) V$$

- Transformer 아키텍처의 핵심인 Self-Attention 메커니즘은 문장 내 연관성이 큰 단어들끼리 비슷하며 학습 가능한 embedding 값을 가지게 해 준다. 이 Attention 행렬의 최종 결과값은 문장 내 각 단어에 문맥이 반영된 새로운 표현임
- $Q = W_q X$, $K = W_k X$, $V = W_v X$, W_q, W_k, W_v : 학습 가능한 가중치
- 행렬의 차원
 - $QK^T = (n \times d_q)(d_q \times n) = (n \times n)$
 - $QK^T V = (n \times n)(n \times d_v) = (n \times d_v)$
 - 이를 계산하는데 소요되는 메모리는 asymptotically $(n \times n) = n^2 \rightarrow$ 입력 토큰 시퀀스의 길이가 길어지면 메모리 사용량이 크게 증가



Multi-Head Attention

Which **do** **you** like better, coffee or tea?

- 문장 타입에 집중하는 어텐션

Which do **you** like better, **coffee** or **tea**?

- 명사에 집중하는 어텐션

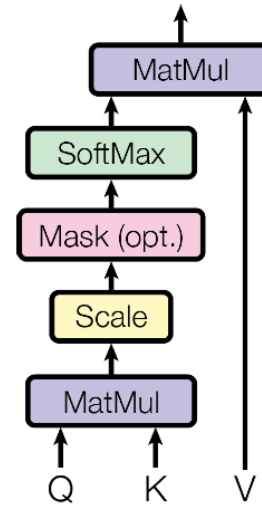
Which do you like better, **coffee** or tea?

- 관계에 집중하는 어텐션

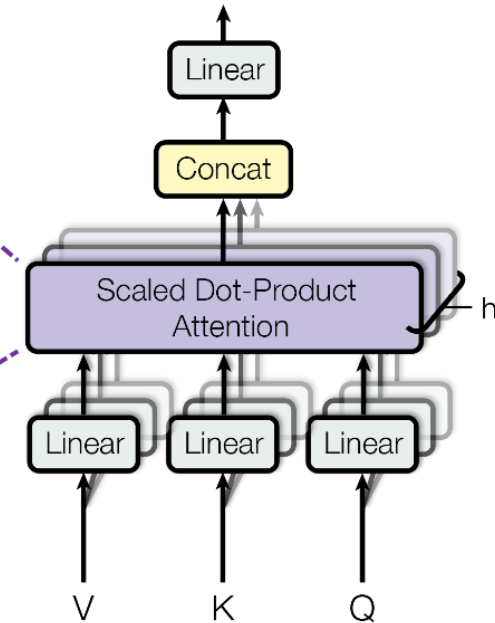
Which do you **like** **better**, coffee or tea?

- 강조에 집중하는 어텐션

Scaled Dot-Product Attention



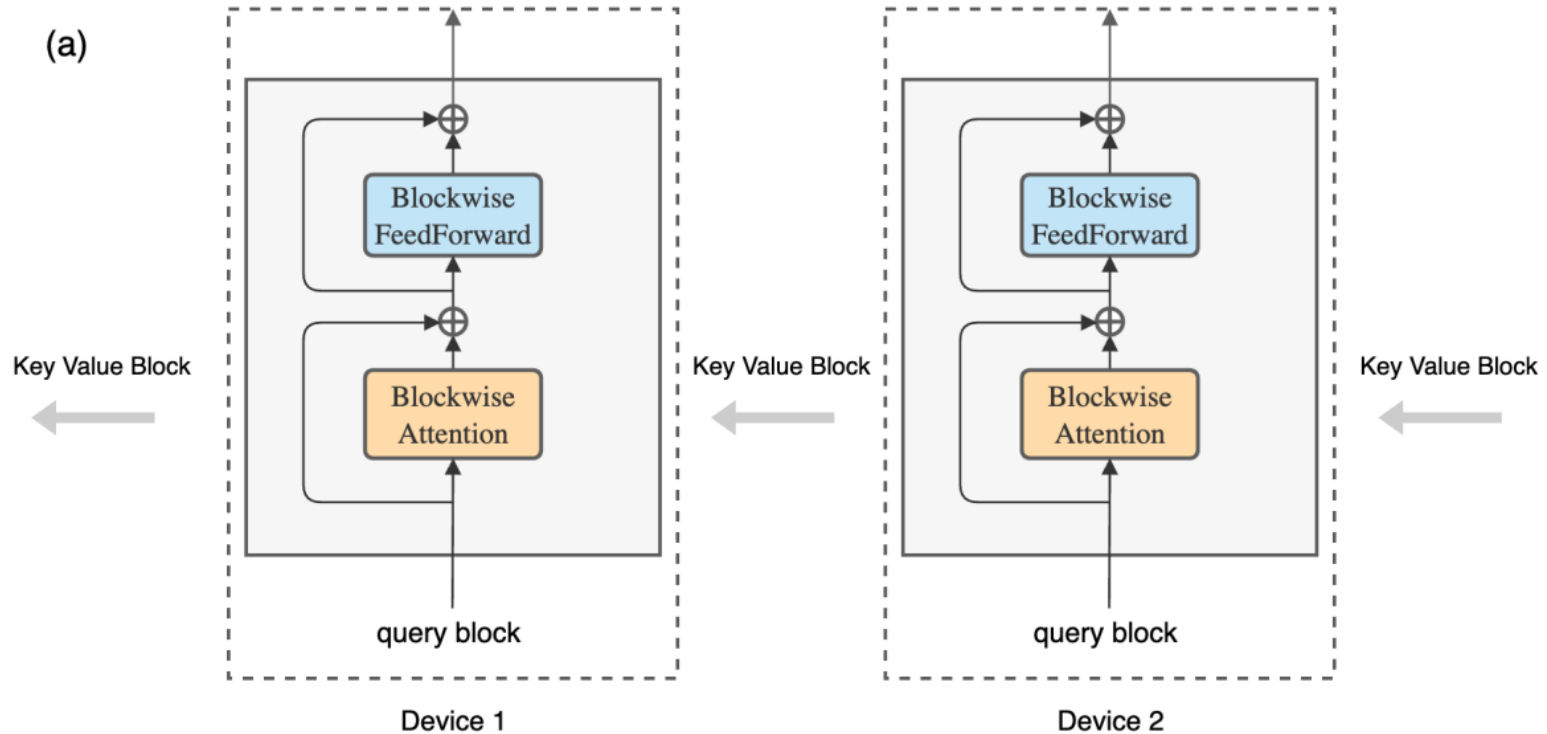
Multi-Head Attention



좌: 기존 Attention, 우: Multi-Head Attention

- Self-attention을 여러 개의 attention head로 나누어 동시에 계산
- 각각 다른 가중치 W_Q , W_K , W_V 를 가지고 학습하므로 각 attention head마다 문장의 다른 의미를 attention 행렬에 담을 수 있음 → Transformer 아키텍처 안에서 텍스트의 표현이 더욱 풍부해짐
- 전체 메모리 사용량은 asymptotically $(m \times n \times n)$, where m = Attention Head의 개수

Blockwise Ring Attention



- 인풋을 작은 크기의 block으로 나누어 각각 다른 컴퓨터에서 Self-attention 과 FFN 병렬 연산. 연산 결과의 순서는 중요하지 않음
- 그러므로 작은 크기의 K,V에 대한 연산들을 순서대로 수행하여 기존 Attention과 동일한 결과를 낼 수 있음
- 원형 구조인 Ring 내 각 컴퓨터에게 연산 분배. 그리고 각 컴퓨터는 입력받은 연산만 처리.
- Ring내부 컴퓨터들은 attention K, V를 이전 컴퓨터로부터 받으면서 QK^TV 연산을 수행하고 **동시에** 다음 컴퓨터로 K, V를 보냄 → 병목현상이 없음

Blockwise Ring Attention

- Blockwise attention에는 크게 2가지 연산이 필요함
- 이 연구에서 s : 인풋 토큰 배열의 길이 (이전에는 n 으로 나타냄)를 의미함
- d : hidden size, c : block size (각 컴퓨터 마다 한 번에 처리할 토큰의 개수), $c = s / \text{GPU 개수}$
 - **communication**: 컴퓨터 끼리 K, V 를 주고 받는 연산
 - 메모리 크기 = $4cd$ bytes (각 K, V 의 크기 $2cd$)
 - **computation**: 각 컴퓨터 내부에서 QK^TV 를 구하는 연산
 - 연산횟수 = $4dc^2$ FLOPS
 - QK^T : $(c \times d)(d \times c) \rightarrow (c \times c)$ 를 구한 다음 QK^TV 를 구해야 하므로 $2 \times 2dc^2 = 4dc^2$, 각 숫자가 곱해진 다음 더해지므로 scaler 2

Blockwise Ring Attention

- computation은 더 이상 최적화가 어려우므로 communication 부분 최적화가 목적
- 이 연구의 목표는 **computation-communication overlap** 달성, 즉 항상 전송/수신하는 동시에 연산이 진행되도록 하는 것
- 그러므로 계산 시간 $\geq$ 전송 시간, $\frac{4dc^2}{F} \geq \frac{4cd}{B} \rightarrow c \geq \frac{F}{B}$, where F : 컴퓨터의 처리 성능 $\left(\frac{Flops}{second}\right)$ and B : 대역폭 $\left(\frac{bytes}{second}\right)$ are constants
- 즉 block size c 를 F/B 보다 크게 정한다면 계산하는 동안 전송이 이루어지기 때문에 **computation-communication overlap** 달성이 가능

Blockwise Ring Attention

Spec Per Host	FLOPS	HBM	Interconnect Bandwidth	Minimal Blocksize	Minimal Sequence Len
	(TF)	(GB)	(GB/s)	($\times 1e3$)	($\times 1e3$)
A100 NVLink	312	80	300	1.0	6.2
A100 InfiniBand	312	80	12.5	24.5	149.5
TPU v3	123	16	112	1.1	6.6
TPU v4	275	32	268	1.0	6.2
TPU v5e	196	16	186	1.1	6.3

- 처리 속도 F: Flops/second, 대역폭 B: bytes/second 가 주어졌을 때
- **Computation-communication overlap** 달성을 위한 **block size c** = F/B 의 값들

GPU VRAM 사용량 비교

Layer Type	Self-Attention	FeedForward	Total
Vanilla	$2bns^2$	$8bsh$	$2bhs^2$
Memory efficient attention	$2bsh + 4bch$	$8bsh$	$8bsh$
Memory efficient attention and feedforward	$2bsh$	$2bsh$	$2bsh$
Ring Attention	$6bch$	$2bch$	$6bch$

- b: batch size, h: hidden dimension, n: number of attention heads, s: sequence length(input sequence 의 토큰 개수) , c: block size (한번에 처리되는 토큰 개수)
- 기존 attention은 메모리 요구 사항이 매우 큼 (s^2 , sequence length에 제곱 비례)
- 하지만 Blockwise Ring Attention은 sequence length s를 여러 개 노드로 나누어 계산하기 때문에 $2bns^2 > 6bch$ 으로 각 VRAM 요구사항이 훨씬 줄어듦
- Blockwise RingAttention은 Flash Attention에 기반하므로 각 노드에서 연산 시 c^2 의 VRAM을 사용하지 않음
 - 느린 VRAM대신 연산속도가 빠른 GPU 내부 SRAM에서 QK^T 를 나누어 계산 후 VRAM에 결과값만 저장
- 하나의 컴퓨터는 한번에 하나의 block 처리. Q가 모든 block에 대하여 KV와 연산이 완료 되었을 때 softmax와 scaling 적용

GPU VRAM 사용량 비교

Layer Type	Self-Attention	FeedForward	Total
Vanilla	$2bns^2$	$8bsh$	$2bhs^2$
Memory efficient attention	$2bsh + 4bch$	$8bsh$	$8bsh$
Memory efficient attention and feedforward	$2bsh$	$2bsh$	$2bsh$
Ring Attention	$6bch$	$2bch$	$6bch$

- 예를 들어, sequence length = 128,000, $b = 1$, $h = 1024$ 일 때 기존 attention 은 QK^T 계산 시 s^2 의 메모리가 필요하기 때문에 약 **32.8 GB**의 VRAM이 필요했지만,
 - Blockwise Ring Attention 적용 시 8개 GPU 사용, C (block size) = $128,000 / 8 = 16,000$, $h = 1024$ 일 때 Q, K, V행렬을 8개로 나누어 계산하므로 각 노드마다 약 **197 MB** 의 VRAM만 필요
- sequence length, context window가 획기적으로 늘어날 수 있음

References

Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Aspell, A., Mishkin, P., Clark, J., Krueger, G., & Sutskever, I. (2021). *Learning Transferable Visual Models From Natural Language Supervision*. Proceedings of the 38th International Conference on Machine Learning, PMLR 139, 8748-8763.

Li, J., Li, D., Savarese, S., & Hoi, S. (2022). *BLIP: Bootstrapping Language-Image Pre-training for Unified Vision-Language Understanding and Generation*. Proceedings of the 39th International Conference on Machine Learning, PMLR 162, 12763-12779.

Esser, P., Rombach, R., & Ommer, B. (2021). *Taming Transformers for High-Resolution Image Synthesis*. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 19868-19878.